

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 12:

Archivos I

Contenido

1. ¿Qué sucede cuando cerramos un programa?
2. Streams en C++: Concepto
3. Clases para el manejo de archivos
4. Ciclo de vida de un archivo
5. Archivos de texto: Lectura, Escritura
6. Modos de apertura de un archivo
7. Estados del stream
8. Ejercicio integrador

1. ¿Qué sucede cuando cerramos un programa?

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string nombres[3] = {"Ana", "Luis", "María"};
    double notas[3]   = {17.5, 14.0, 19.0};
    int n = 3;

    cout << "Registro de Notas" << endl;
    for (int i = 0; i < n; i++)
        cout << nombres[i] << ": " << notas[i] << endl;
    return 0;
}
```

¿Dónde quedaron las notas de Ana, Luis y María?

Se perdieron, la RAM se liberó, no quedan registros

Consecuencia: El problema real

- Registrar notas de 600 estudiantes durante el examen final. El servidor se reinicia por un corte de luz.
  - ✓ Sin archivos: las 600 notas desaparecen
  - ✓ Con archivos: el sistema las recupera en segundos

Sin archivos	Con archivos
Los datos viven solo mientras el programa corre	Los datos sobreviven al cierre del programa
Cada ejecución empieza desde cero	Cada ejecución puede continuar donde quedó
Imposible compartir datos entre programas	Otros programas pueden leer los mismos datos
Imposible manejar más datos que la RAM	Limitado solo por el disco

La solución: uso de archivos en C++

```cpp
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    string nombres[3] = {"Ana", "Luis", "María"};
    double notas[3]   = {17.5, 14.0, 19.0};
    int n = 3;

    ofstream archivo("notas.txt");

    for (int i = 0; i < n; i++)
        archivo << nombres[i] << " " << notas[i] << endl;
    archivo.close();

    cout << "Datos guardados en notas.txt" << endl;

    return 0;

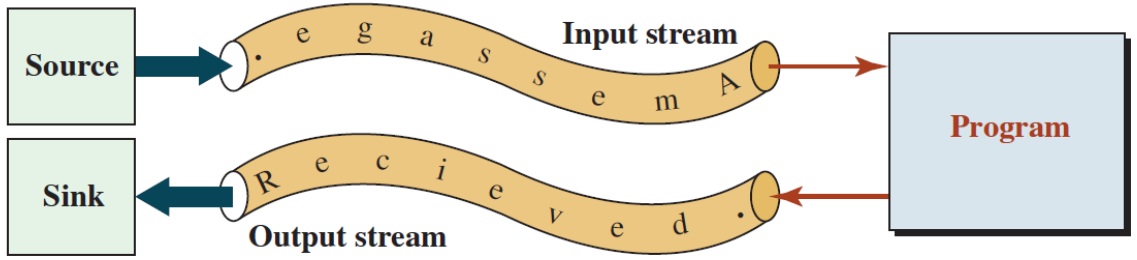
}
```



2. ¿Qué es un Stream?

Un **stream** es una secuencia de bytes que fluye desde una fuente hacia un destino en un programa

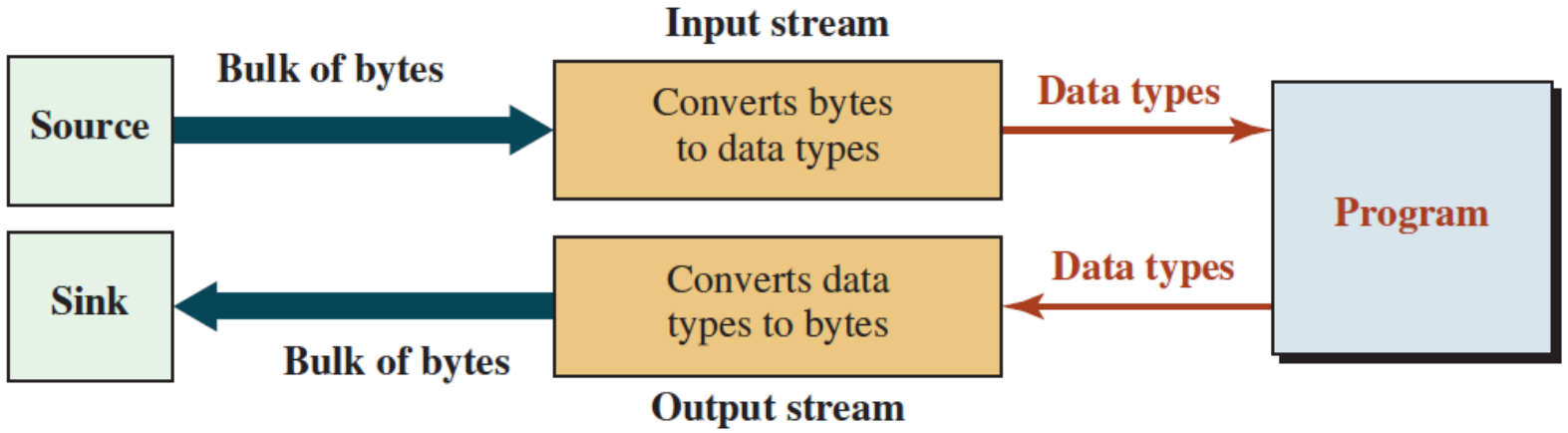
1. **Es un mediador** — la fuente y el destino no se conectan directamente al programa. El stream se encarga de la comunicación entre ellos.
2. **Convierte datos** — la memoria trabaja en binario. El teclado y la pantalla trabajan en texto. El stream traduce entre ambos mundos automáticamente.
3. **Usa un búfer** — los datos no se mueven byte a byte sino en bloques. El búfer acumula los bytes y los entrega en bloque, lo cual es mucho más eficiente.



¿Cómo viajan los datos?

Toda fuente y destino de datos maneja bytes. Pero hay dos formas de interpretar esos bytes: como **texto** o como **binario**.

Fuente/Destino	Modo	Ejemplo
Teclado / Pantalla	Solo texto	El número `42` viaja como los caracteres `'4'` y `'2'`
Archivos	Texto o binario	Tú decides al abrir el archivo
Strings (`stringstream`)	Solo texto	Útil para conversiones en memoria



¿Por qué existe el búfer?

¿Cómo movemos 1000 ladrillos de un punto A a un punto B ?

- Podrías ir y volver 1000 veces cargando un ladrillo cada vez, o
- Podrías usar una carretilla, cargarla con 50 ladrillos, y hacer solo 20 viajes.

El **búfer es la carretilla**. Acceder al disco o al teclado byte a byte sería ineficiente. El búfer acumula datos y los envía en bloques:

```
ofstream archivo("notas.txt");
archivo << "Ana: 17.5" << endl; // ¿Ya está en el disco? No necesariamente.
                               // Puede estar en el búfer esperando ser enviado.

archivo.close(); // vacía el búfer automáticamente
                 // alternativamente archivo.flush();

// Por ello se recomienda cerrar el archivo explícitamente
// antes de leer lo que acabas de escribir.
```

3. Clases para el manejo de archivos en C++

Los objetos **cin** y **cout** pertenecen a clases de stream que el sistema crea automáticamente. Para archivos, nosotros creamos los objetos y debemos elegir la clase correcta:

ifstream (input file stream) Para leer desde un archivo. El archivo debe existir.

```
ifstream archivo("notas.txt"); // abrir para leer
```

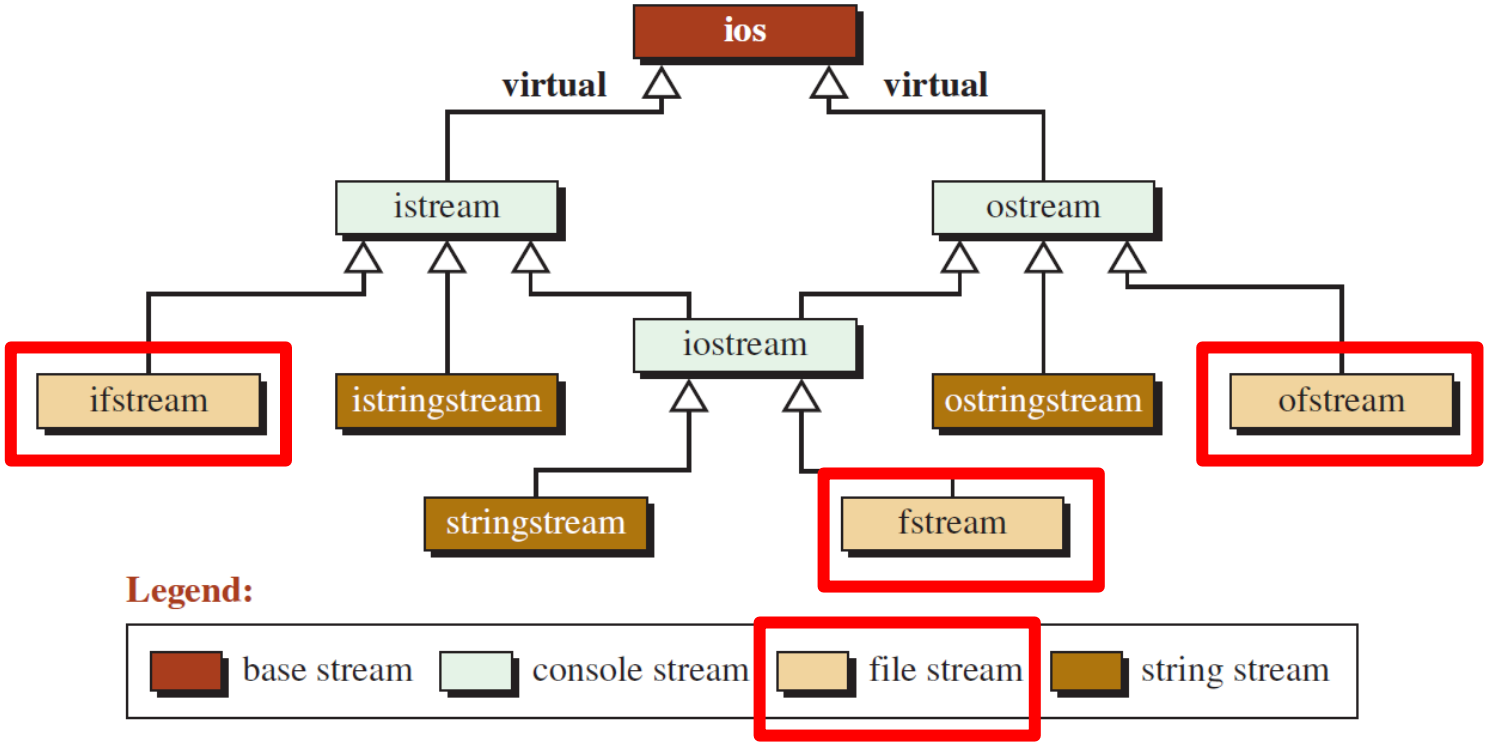
ofstream (output file stream) Para escribir en un archivo. Si no existe, lo crea. Si existe, lo sobreescribe por defecto

```
ofstream archivo("notas.txt"); // abrir para escribir
```

fstream (file stream) Para leer y escribir en el mismo archivo. Más flexible, pero requiere especificar los modos explícitamente.

```
fstream archivo("notas.txt", ios::in | ios::out); // leer y escribir
```

¿De dónde vienen estas clases?



1) **La raíz es ios**, todas las clases heredan de ios. Esta clase define los estados del stream, los modos de apertura y las banderas de formato. No se puede instanciar directamente.
2) **ifstream hereda de istream, igual que cin** Por eso puedes usar:
   `getline(archivo, linea);` solo cambia la fuente de datos.
3) **ofstream hereda de ostream, igual que cout:** Por eso puedes escribir:
   `archivo << nombre << endl;` solo cambia el destino.

4. Ciclo de vida de un archivo

- **Paso 1 Abrir.** Crear el objeto de stream y conectarlo al archivo en disco.

```
ifstream archivo("notas.txt");
```

- **Paso 2 Verificar** que el archivo se abrió correctamente antes de usarlo.

```
if (!archivo.is_open()) {
     cerr << "Error: no se pudo abrir el archivo." << endl;
     return 1;
 }
```

- **Paso 3 Leer o Escribir.** Operar sobre el archivo solo si la verificación fue exitosa.

```
string linea;
 while (getline(archivo, linea)) {
     cout << linea << endl;
 }
```

- **Paso 4 Cerrar** Liberar el recurso explícitamente.

```
archivo.close();
```

Estos cuatro pasos son el contrato mínimo con el SO. El archivo es un recurso externo — no es tuyo hasta que lo abres, y debes devolverlo cuando terminas.

¿Qué pasa si olvidamos cerrar?

- En C++, cuando un objeto de stream sale de su ámbito, su destructor se ejecuta automáticamente y cierra el archivo (como vimos, esto es, RAII)

- Pero close() explícito sigue siendo buena práctica:
  Al cerrar explícitamente podemos:
  a. detectar errores de escritura.

```
archivo.close();
if (archivo.fail()) {
     cerr << "Error al cerrar: datos pueden estar incompletos." << endl;
}
```

  b. reutilizar el objeto en el mismo ámbito

```
fstream archivo("datos.txt", ios::out);
archivo << "primera escritura";
archivo.close();
```

```
// reutilizar el mismo objeto
archivo.open("datos.txt", ios::in);
string linea;
getline(archivo, linea);
```

5. Archivos de texto: Lectura y Escritura

- Trabajar con archivos implica abrir, verificar, procesar y cerrar:
  1. Crear el stream y asociarlo al archivo (ifstream, ofstream, fstream)
  2. Verificar que la apertura fue exitosa
  3. Leer o escribir datos
  4. Cerrar el archivo

- La lectura depende del formato de los datos:

Situación	Usar
Datos separados por espacios, sin espacios internos	`>>`
Datos con espacios internos (nombres, frases)	`getline`
Línea con múltiples tipos de datos mezclados	`getline` + `find`/`substr`

Streams y funciones

- Los streams son objetos no copiables; por ello se pasan a las funciones mediante referencias (&).

- Lectura:

```
void mostrarNotas(ifstream& archivo) {
    string nombre;
    double nota;
    while (archivo >> nombre >> nota)
        cout << nombre << ": " << nota << endl;
}
```

```
ifstream archivo("notas.txt");
mostrarNotas(archivo);
archivo.close();
```

- Escritura:

```
void guardarNota(ofstream& archivo,
                 string nombre,
                 double nota) {
    archivo << nombre << " “ << nota << endl;
}
```

```
ofstream archivo("notas.txt");
guardarNota(archivo, "Ana", 17.5);
guardarNota(archivo, "Luis", 14.0);
archivo.close();
```

6. Modos de apertura de archivos

Modo	Significado	¿Cuándo usarlo?	Ejemplo
`ios::in`	Abrir para lectura	Leer un archivo existente	`ifstream f("datos.txt")`
`ios::out`	Abrir para escritura, destruye contenido	Crear o reemplazar un archivo	`ofstream f("datos.txt")`
`ios::app`	Agregar al final sin destruir contenido	Agregar registros a un log o historial	`ofstream f("log.txt", ios::app)`
`ios::ate`	Abrir y moverse al final inmediatamente	Leer el final o agregar con control manual	`fstream f("datos.txt", ios::ate)`
`ios::trunc`	Truncar a longitud cero al abrir	Limpiar un archivo antes de escribir	`fstream f("datos.txt", ios::trunc)`
`ios::binary`	Modo binario en lugar de texto	Guardar structs, números exactos	`ofstream f("datos.bin", ios::binary)`

Si no se especifica el modo de apertura, **cada clase usa su modo por defecto**: **ifstream usa ios::in** por defecto. **ofstream usa ios::out | ios::trunc** por defecto (es decir, borra el contenido existente). **fstream no tiene modo por defecto** (siempre debes especificarlo)

Combinaciones más usadas en la práctica

Los modos se combinan con el operador | (OR a nivel de bits):

- **Agregar al final de un archivo existente:**

```
ofstream log("historial.txt", ios::app);
log << "Sesión iniciada: " << fecha << endl;
```

- **Leer y escribir sin destruir el contenido:**

```
fstream archivo("datos.txt", ios::in | ios::out);
```

- **Leer y escribir destruyendo el contenido previo:**

```
fstream archivo("datos.txt", ios::in | ios::out | ios::trunc);
```

- **Archivo binario para lectura y escritura** (siguiente sesión):

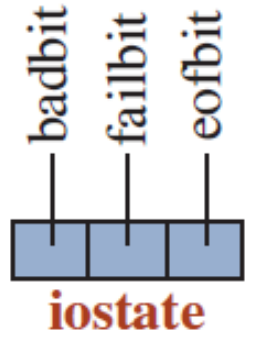
```
fstream archivo("datos.bin", ios::in | ios::out | ios::binary);
```

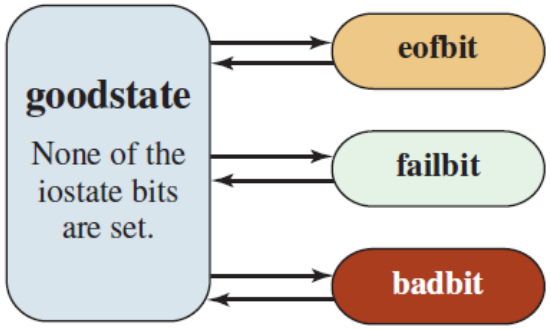
Advertencia: Siempre verifica el modo antes de abrir un archivo que ya existe, recuerda que `ofstream f("datos.txt")` sobreescribe el archivo existente

7. Estados del stream: ¿Cómo saber si algo salió mal?

- Un stream lleva registro de su salud interna mediante banderas. Cuando ocurre un error, una o más de estas banderas se activan y el stream deja de funcionar hasta que las limpies.

Bandera	Cuándo se activa	Causa más común
`goodbit`	Todo bien, ninguna bandera activa	Estado normal de operación
`eofbit`	Se llegó al final del archivo	Leer más allá del último byte
`failbit`	Operación de I/O falló	Leer un `int` donde hay texto
`badbit`	Error grave, integridad del stream perdida	Disco lleno, archivo corrupto





Importante: cuando failbit o badbit están activos, todas las operaciones subsiguientes sobre ese stream se ignoran silenciosamente. El stream queda bloqueado hasta que limpies las banderas.

Recuperando un stream fallido: clear() e ignore()

- ¿Por qué ocurre esto?
  El operador >> consume el número pero deja el salto de línea ‘\n’ en el búfer. Cuando getline() llega, encuentra ese ‘\n’ y cree que ya terminó la línea, devuelve una cadena vacía

- clear() sin argumentos limpia todas las banderas y devuelve el stream a goodbit. Sin clear(), las llamadas a ignore() también se ignoran porque el stream está bloqueado.

- El orden siempre es:

```
cin.clear();
cin.ignore();
```

8. Ejercicio integrador: La agenda que no olvida

Desarrolla un programa de agenda de contactos con las siguientes funciones:

- Agregar un contacto (nombre y teléfono) guardándolo en un archivo
- Mostrar todos los contactos guardados
- Buscar un contacto por nombre

El programa debe persistir los datos entre ejecuciones. Al cerrarlo y volverlo a abrir, los contactos deben seguir ahí.

- Usar ofstream con ios::app para agregar sin borrar
- Usar ifstream con getline correcto para leer
- Verificar siempre si el archivo se abrió correctamente
- Cerrar el archivo explícitamente después de cada operación